Content

이번 주차의 학습목표와 학습내용을 확인하세요.

01. 자료구조와 알고리즘의 정의 및 필요성

자료구조와 알고리즘의 개념을 이해하고 설명할 수 있다.

02. 추상자료형

추상자료형의 개념을 이해하고 설명할 수 있다.

03. 알고리즘의 성능 분석 및 순환 알고리즘

알고리즘의 시간 복잡도와 순환 알고리즘에 대해 설명할 수 있다.

강의소개

왜 자료구조를 배워야 할까?

자료들의 효율적 관리와 구조화를 다루는 분야로
컴퓨터 공학에서 매우 중요함

최근 각광받는 데이터 사이언스 분야의 이해를 위한 선행지식임

강의소개

수업내용

01 리스트, 스택, 큐 등 자료구조 기본개념 습득

02 정렬, 탐색 등을 이용한 알고리즘 활용

수업목표

01 자료구조 개념 이해와 기본 연산 숙지

02 기본 알고리즘 이해와 알고리즘의 복잡성 분석

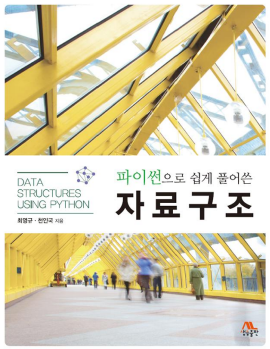
03 파이썬을 이용해 습득한 지식 활용

강의소개

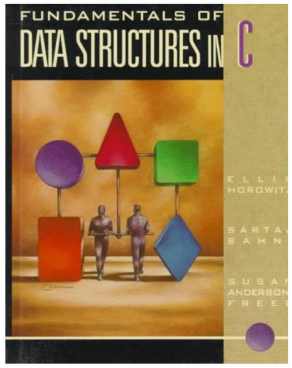
선수지식

파이썬 코드 실행에 대한 기본능력

참고교재



「파이썬으로 쉽게 풀어쓴 자료구조」
최영규, 천인국



「Fundamentals of Data Structures in C」
Ellis Horowitz 외

강의소개

수업운영

강의

개념 이해 및
예제 학습

과제

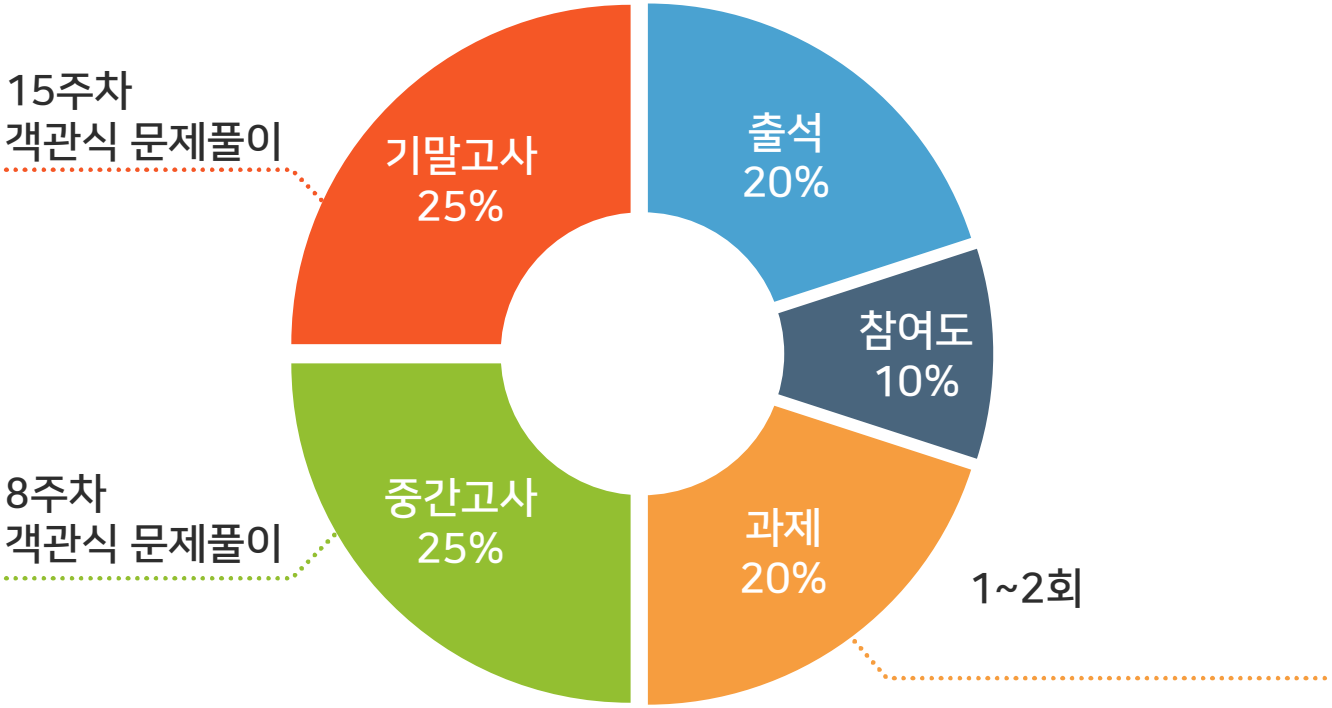
학습내용 확인
및 연습

시험

이해도 측정

강의소개

평가방식



강의소개

주차별 수업계획

1~6주차	리스트, 집합, 스택, 큐, 덱, 연결
7~14주차	정렬, 탐색, 트리, 탐색트리, 그래프, 가중치 그래프
8주차	중간시험
15주차	기말시험

* 수업계획은 일부 변경될 수 있습니다.

자료구조와 알고리즘에 대한
기본개념 습득 및 활용 능력 배양

자료구조와 알고리즘의 정의 및 필요성

자료구조와 알고리즘이란

자료구조의 의미

자료를 정리하고 조직화하는 이유

사물을 편리하고 효율적으로 사용하기 위해
다양한 자료를 효율적인 규칙에 따라 정리하기 위해

컴퓨터에서의 자료구조란?
(Data Structure)

컴퓨터에서 자료를 정리하고 조직화하는 다양한 구조

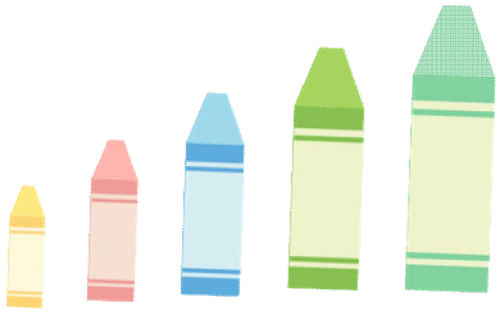
자료구조와 알고리즘이란

자료구조의 방법

선형 자료구조

항목들을 순서대로 나열하여 저장

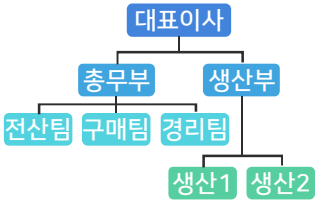
- 리스트 : 자유로운 선형 자료구조
- 스택, 큐, 덱 : 항목의 접근을 전단이나 후단으로 제한

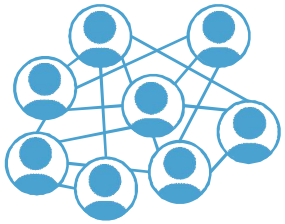


비선형 자료구조

항목들이 복잡하게 연결

- 트리 : 회사 조직도나 폴더와 같은 계층 구조
- 그래프 : 다양한 문제 해결을 위해 복잡한 연결관계를 표현한 구조





자료구조와 알고리즘이란

알고리즘의 의미

컴퓨터로 문제를 풀기 위한 단계적 절차

사전에서 단어는 어떻게 찾나?

프로그램 ➡ 자료구조 ✚ 알고리즘

<이미지 출처: 최명규, 천인국. '파이썬으로 쉽게 풀어쓴 자료구조'. 생능출판사>

자료구조와 알고리즘이란

알고리즘의 조건

입력	0개 이상의 입력 존재
출력	1개 이상의 출력 존재
명백성	각 명령어의 의미는 모호하지 않고 명확해야 함
유한성	한정된 수의 단계 후에 반드시 종료되어야 함
유효성	각 명령어들은 실행 가능한 연산이어야 함

자료구조와 알고리즘이란

알고리즘의 기술 방법

자연어 표현

흐름도 표현

유사코드 표현

특정언어 표현

- 가독성이 좋으나 단어정의가 정확하지 않으면 의미가 모호함

find_max(A)

1. 배열 A의 첫 번째 요소를 변수 tmp에 복사한다.
2. 배열 A의 다음 요소들을 차례대로 tmp와 비교하여, 더 크면 그 값을 tmp로 복사한다.
3. 배열 A의 모든 요소를 비교했으면 tmp를 반환한다.

자료구조와 알고리즘이란

알고리즘의 기술 방법

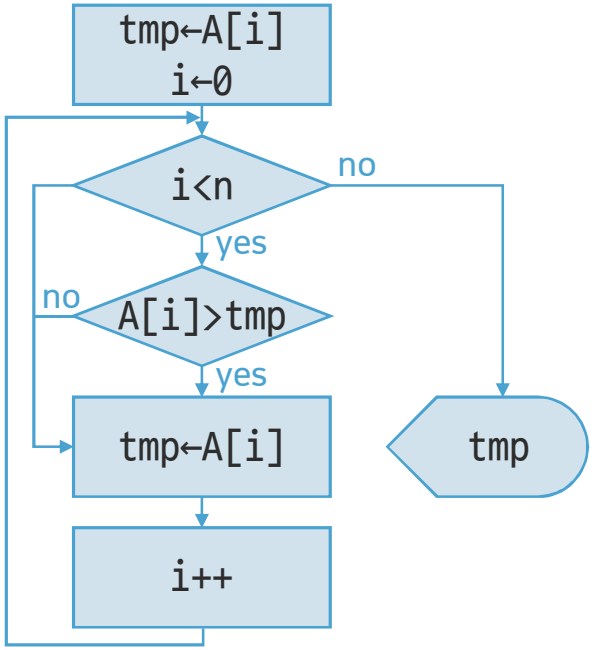
자연어 표현

흐름도 표현

유사코드 표현

특정언어 표현

- 직관적이나 복잡한 알고리즘 기술 시 매우 복잡해질 우려가 있음



자료구조와 알고리즘이란

알고리즘의 기술 방법

자연어 표현

흐름도 표현

유사코드 표현

특정언어 표현

- 알고리즘 핵심내용에 집중하며 프로그램 구현시 문제들을 감출 수 있음

```
find_max(A)
tmp ← A[0];
for i←1 to size(A) do
      if tmp < A[i] then
            tmp ← A[i]
return tmp
```

자료구조와 알고리즘이란

알고리즘의 기술 방법

자연어 표현

흐름도 표현

유사코드 표현

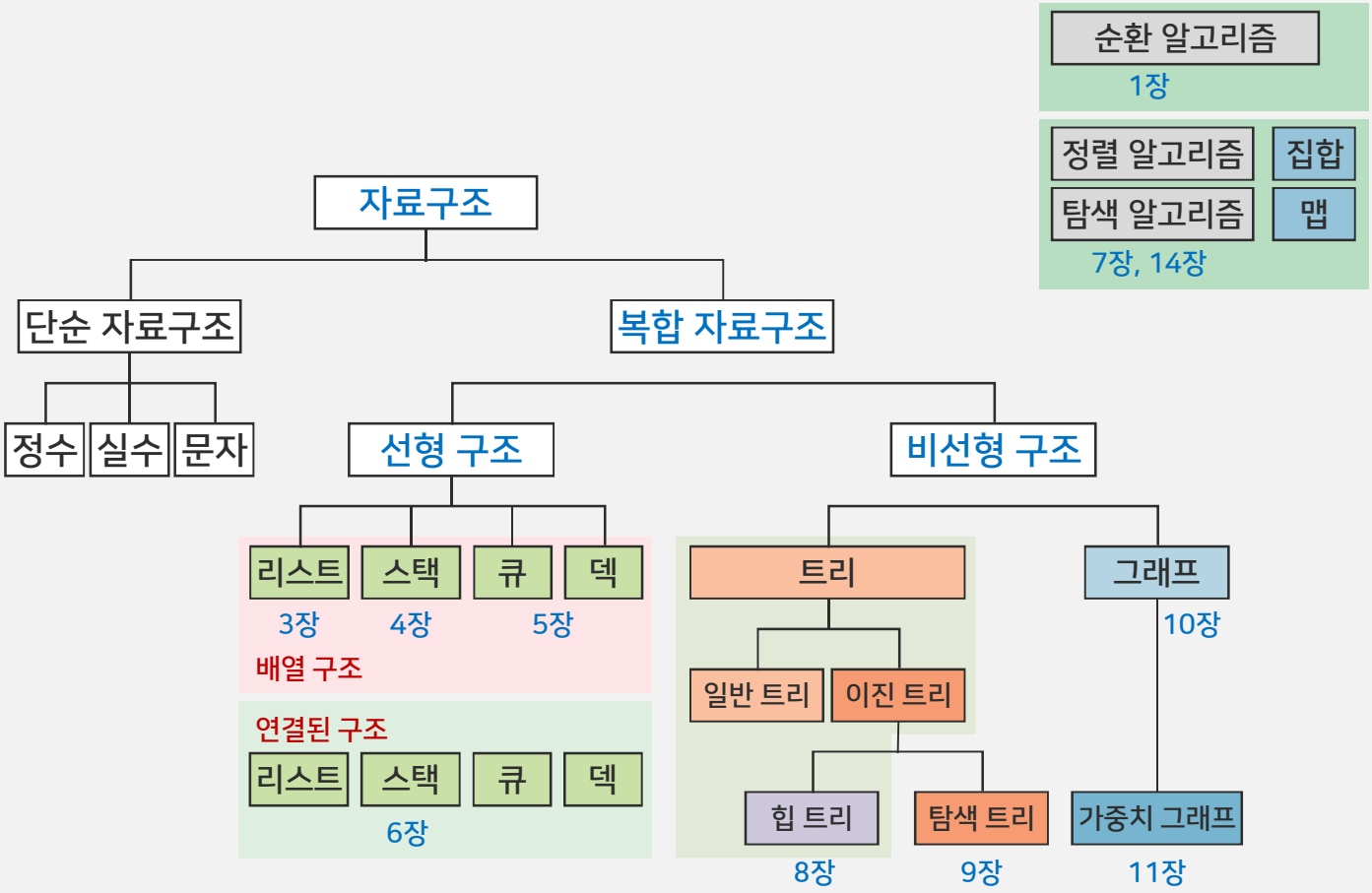
특정언어 표현

- 가장 정확하게 알고리즘을 기술하지만, 알고리즘 핵심내용의 이해를 방해할 수 있음
- 파이썬은 C나 JAVA에 비해 간결한 표현 가능

```
def find_max( A ):
    tmp = A[0]
    for item in A :
        if item > tmp :
            tmp = item
    return tmp
```

자료구조와 알고리즘이란

본 수업 구성(자료구조와 알고리즘)



추상자료형

추상자료형과 자료구조

추상자료형(ADT, Abstract Data Type)

데이터 타입(자료형)을 프로그래머가 추상적(수학적)으로 정의한 것

What

데이터나 연산이
무엇(What)인가를 정의

How

데이터나 연산을 어떻게(How)
구현할 것인지는 정의하지 않음

시스템의 핵심구조나 동작에만 집중

추상자료형과 자료구조

추상자료형(ADT, Abstract Data Type)

예 Bag(가방)의 추상자료형

데이터

- 중복된 항목을 허용하는 자료들의 저장소
- 항목들은 특별한 순서가 없이 개별적으로 저장되지만 항목 간의 비교는 가능해야 함

연산

- `Bag()` : 비어있는 가방을 새로 만듦
- `insert(e)` : 가방에 항목 e를 넣음
- `remove(e)` : 가방에 e가 있는지 검사하여 있으면 이 항목을 꺼냄
- `contains(e)` : e가 들어있으면 True를, 없으면 False를 반환함
- `count()` : 가방에 들어 있는 항목들의 수를 반환함

<이미지 출처: 최명규, 천인국. '파이썬으로 쉽게 풀어쓴 자료구조'. 생능출판사>

추상자료형과 자료구조

추상자료형(ADT, Abstract Data Type)

예 Bag(가방)의 추상자료형의 구현 (파이썬)

```
def contains(bag,e) :        # bag에 항목 e가 있는지 검사하는 함수
    return e in bag          # 파이썬의 in 연산자 사용

def insert(bag, e) :         # bag에 항복 e를 넣는 함수
    bag.append(e)            # 파이썬 리스트의 append메소드 사용

def remove(bag, e) :         # bag에서 항목 e를 삭제하는 함수
    bag.remove(e)            # 파이썬 리스트의 remove메소드 사용

def count(bag) :             # bag의 전체 항목 수를 계산하는 함수
    return len(bag)          # 파이썬의 len 함수 사용
```

추상자료형과 자료구조

추상자료형(ADT, Abstract Data Type)

예 Bag(가방)의 추상자료형을 활용한 자료관리

```
myBag = []                          # Bag을 위한 빈 리스트를 만듦
insert(myBag, '휴대폰')             # Bag에 휴대폰 삽입
insert(myBag, '지갑')               # Bag에 지갑 삽입
insert(myBag, '손수건')             # Bag에 손수건 삽입
insert(myBag, '빗')                 # Bag에 빗 삽입
insert(myBag, '자료구조')           # Bag에 자료구조 삽입
insert(myBag, '야구공')             # Bag에 야구공 삽입
print('가방속의 물건:', myBag)      # Bag의 내용 출력
```

```
C:\WINDOWS\system32\cmd.exe
내 가방속의 물건: ['휴대폰', '지갑', '손수건', '빗', '자료구조', '야구공']
```

추상자료형과 자료구조

추상자료형(ADT, Abstract Data Type)

예 Bag(가방)의 추상자료형을 활용한 자료관리

```
insert(myBag, '빗')                # Bag에서 '빗' 삽입(중복)
remove(myBag, '손수건')            # Bag에서 '손수건' 삭제
print('가방속의 물건:', myBag)      # Bag의 내용 출력
```

C:\WINDOWS\system32\cmd.exe

```
내 가방속의 물건: ['휴대폰', '지갑', '손수건', '빗', '자료구조', '야구공']
내 가방속의 물건: ['휴대폰', '지갑', '빗', '자료구조', '야구공', '빗']
```

알고리즘의 성능 분석 및 순환 알고리즘

복잡도 분석

알고리즘 성능 분석 방법

실행시간 측정

- 두 개의 알고리즘의 실제 **실행 시간을 측정 비교**
- 동일한 하드웨어 환경에서 구현해야 함

복잡도 분석

- 직접 실행하지 않고 알고리즘의 **연산 횟수를 계산**
- 일반적으로 연산의 횟수는 n의 함수임

시간 복잡도 ▶ 수행 시간

공간 복잡도 ▶ 메모리 공간

복잡도 분석

실행시간 측정 방법

```
import time                          # time 모듈 불러오기

myBag = []                           # 비어있는 새로운 가방을 하나 만듦
start = time.time()                  # 현재 시각을 start에 저장
insert(myBag, '축구공')              # 실행시간을 측정하려는 코드
...                                  # ...
end = time.time()                    # 현재 시각을 end에 저장
print("실행시간 = ", end-start)       # 실행시간(종료-시작)을 출력
```

복잡도 분석

복잡도 분석 방법

시간 복잡도

- 산술, 대입, 비교, 이동의 기본적인 연산 고려
- 알고리즘 수행에 필요한 연산의 개수를 계산

입력 개수
n에 대한 함수

→

시간 복잡도 함수,
T(n)

복잡도 분석

복잡도 분석 방법

Bag 삽입연산 복잡도 비교

리스트의 맨 뒤에 삽입

- 효율적임
- 바로 삽입 가능함

```
def insert(bag, e) :        # bag에 항목e를 넣는 함수
    bag.append(e)           # 파이썬 리스트의 맨 뒤에 추가
```

리스트의 맨 앞에 삽입

- 비효율적임
- 가방의 모든 물건을 이동한 후 삽입해야 함

```
def insert(bag, e) :        # bag에 항목e를 넣는 함수
    bag.insert(0, e)        # 파이썬 리스트의 맨 앞에 추가
```

복잡도 분석

복잡도 분석 방법

3가지 알고리즘 연산의 수 계산하기

알고리즘	A	B	C

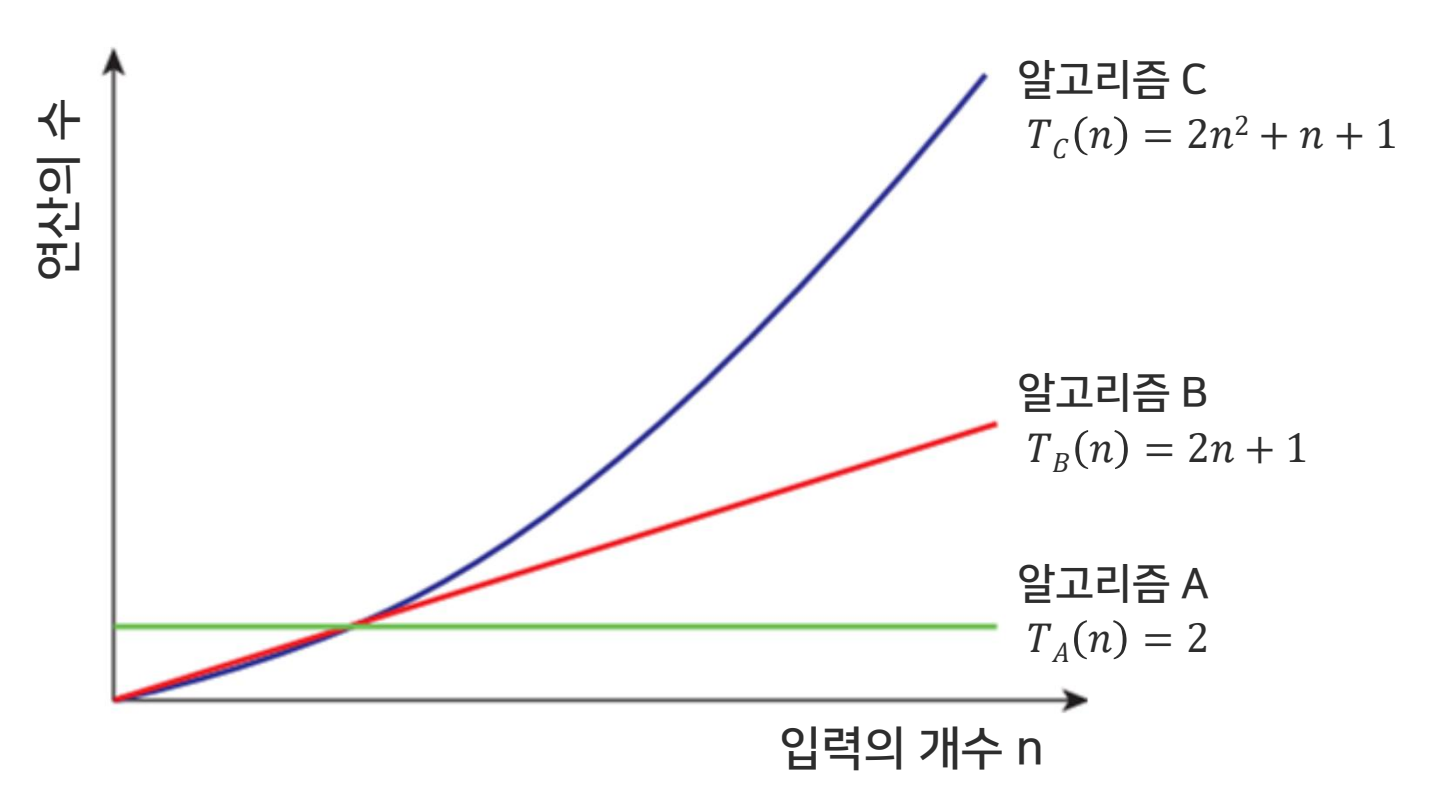
유사 코드	sum ← n*n	for i←1 to n do
		  sum ← sum + n
			for i←1 to n do
			  for j←1 to n do
			  sum ← sum + 1

연산 횟수	대입연산 : 1 곱셈연산 : 1	대입연산 : n+1 덧셈연산 : n	대입연산 : $n^2 + n + 1$ 덧셈연산 : n^2
복잡도 함수	$T_A(n) = 2$	$T_B(n) = 2n + 1$	$T_C(n) = 2n^2 + n + 1$

복잡도 분석

복잡도 분석 방법

3가지 알고리즘 연산의 수 비교



빅오

빅오 표기법

1 차수가 가장 큰 항이 절대적인 영향을 줌

➡ 다른 항들은 상대적으로 무시됨

예 $T(n) = n^2 + n + 1$

n=1	$T(n) = 1 + 1 + 1 = 3$ (n^2항이 33.3%)
n=10	$T(n) = 100 + 10 + 1 = 111$ (n^2항이 90%)
n=100	$T(n) = 10000 + 100 + 1 = 10101$($n^2$항이 99%)
n=1,000	$T(n) = 1000000 + 1000 + 1 = 1001001$($n^2$항이 99.9%)

n=100인 경우, $T(n) = n^2 + n + 1$

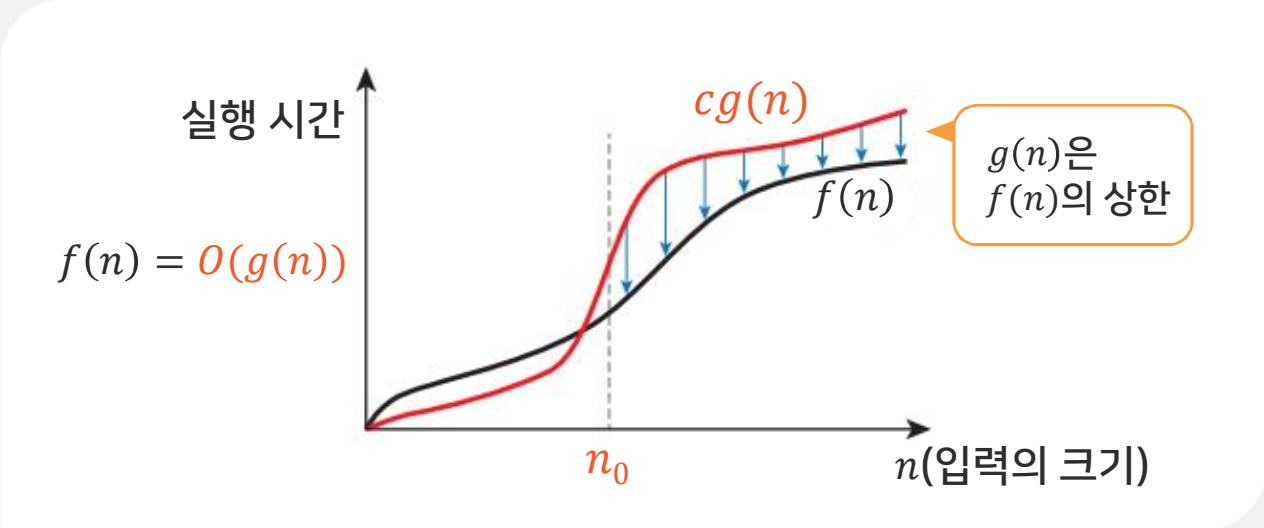
99% 1%

빅오

빅오 표기법

2 연산의 횟수를 대략적(점근적)으로 표기함

- 두 개의 함수 $f(n)$과 $g(n)$이 주어졌을 때 모든 $n > n_0$에 대해 $|f(n)| \leqq c|g(n)|$을 만족하는 상수 c와 n_0가 존재하면 $f(n) = O(g(n))$임



빅오

빅오 표기법 예제

01

$f(n) = 5$ ➡ $O(1)$

$n_0 = 1,\ c = 10$일 때, $n \geq 1$에 대하여 $5 \leq 10 \cdot 1$

02

$f(n) = 2n + 1$ ➡ $O(n)$

$n_0 = 2,\ c = 3$일 때, $n \geq 2$에 대하여 $2n + 1 \leq 3 \cdot n$

03

$f(n) = 3n^2 + 100$ ➡ $O(n^2)$

$n_0 = 100,\ c = 5$일 때, $n \geq 100$에 대하여 $3n^2 + 100 \leq 5 \cdot n^2$

04

$f(n) = 5 \cdot 2^n + 10n^2 + 100$ ➡ $O(2^n)$

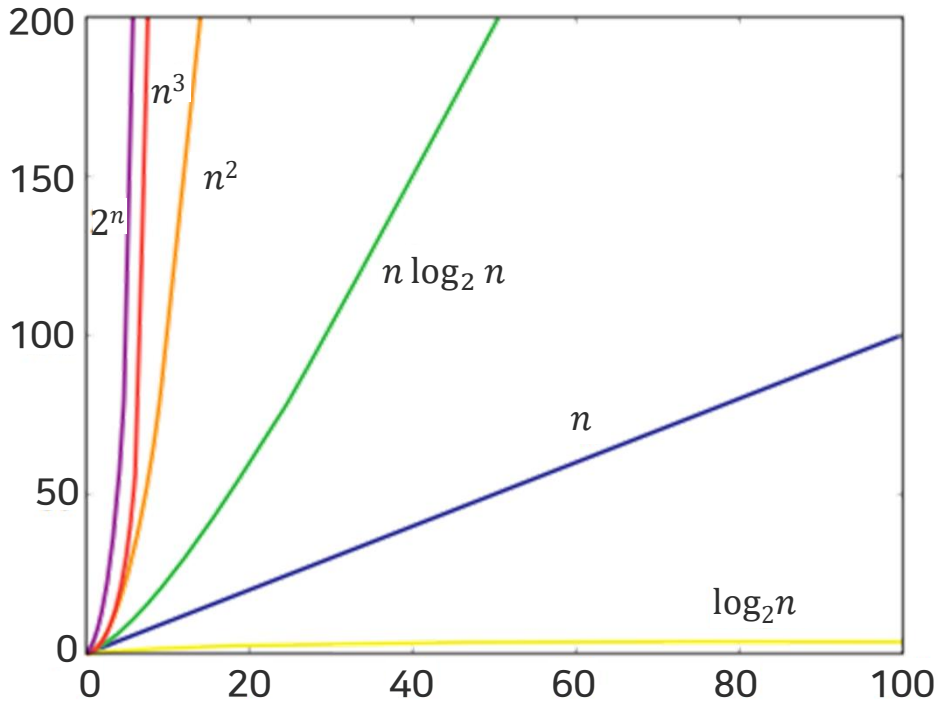
$n_0 = 1000,\ c = 10$일 때,

$n \geq 1000$에 대하여 $5 \cdot 2^n + 10n^2 + 100 < 10 \cdot 2^n$

빅오

빅오 표기법 종류

- $O(1)$: 상수형
- $O(\log n)$: 로그형
- $O(n)$: 선형
- $O(n \log n)$: 선형로그형
- $O(n^2)$: 2차형
- $O(n^3)$: 3차형
- $O(2^n)$: 지수형
- $O(n!)$: 팩토리얼형



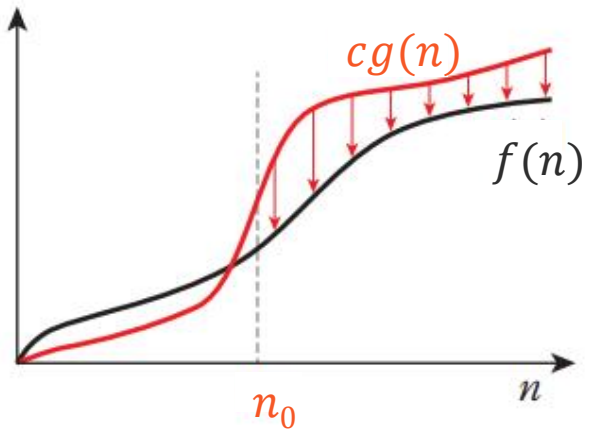
빅오

빅오메가와 빅세타

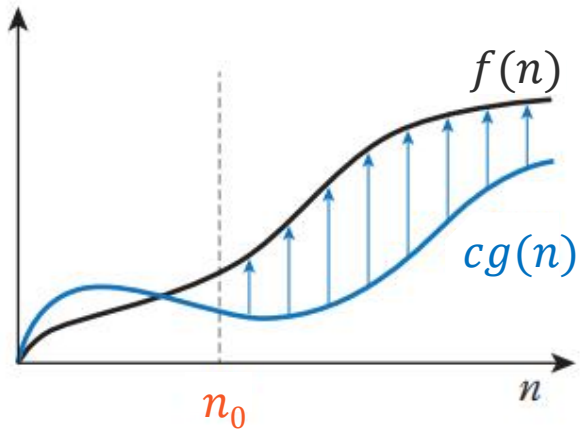
빅오메가 표기법

두 개의 함수 $f(n)$과 $g(n)$이 주어졌을 때 모든 $n > n_0$에 대해
$|f(n)| \geqq c|g(n)|$을 만족하는 상수 c와 n_0가 존재하면
$f(n) = \Omega(g(n))$임

$f(n) = O(g(n))$



$f(n) = \Omega(g(n))$



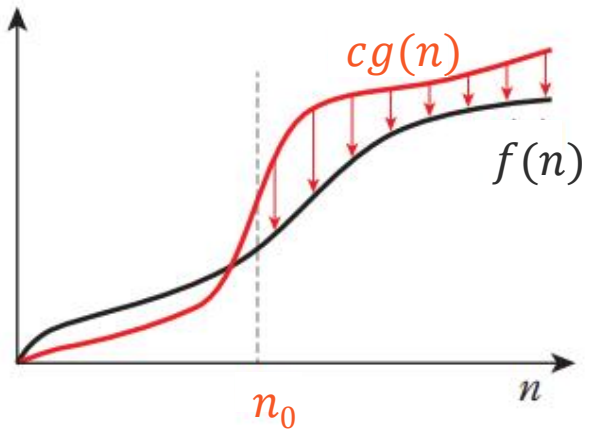
빅오

빅오메가와 빅세타

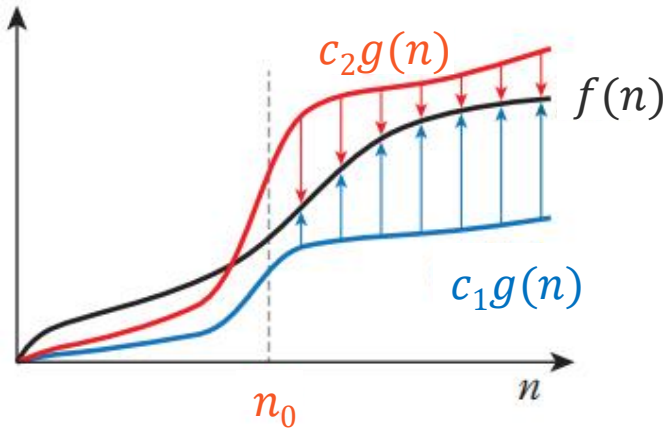
빅세타 표기법

두 개의 함수 $f(n)$과 $g(n)$이 주어졌을 때 모든 $n > n_0$에 대해
$c_1|g(n)| \leqq f(n) \leqq c_2|g(n)|$을 만족하는 상수 c_1, c_2와 n_0가 존재하면
$f(n) = \Theta(g(n))$임

$f(n) = O(g(n))$



$f(n) = \Theta(g(n))$



빅오

최선, 평균, 최악의 경우

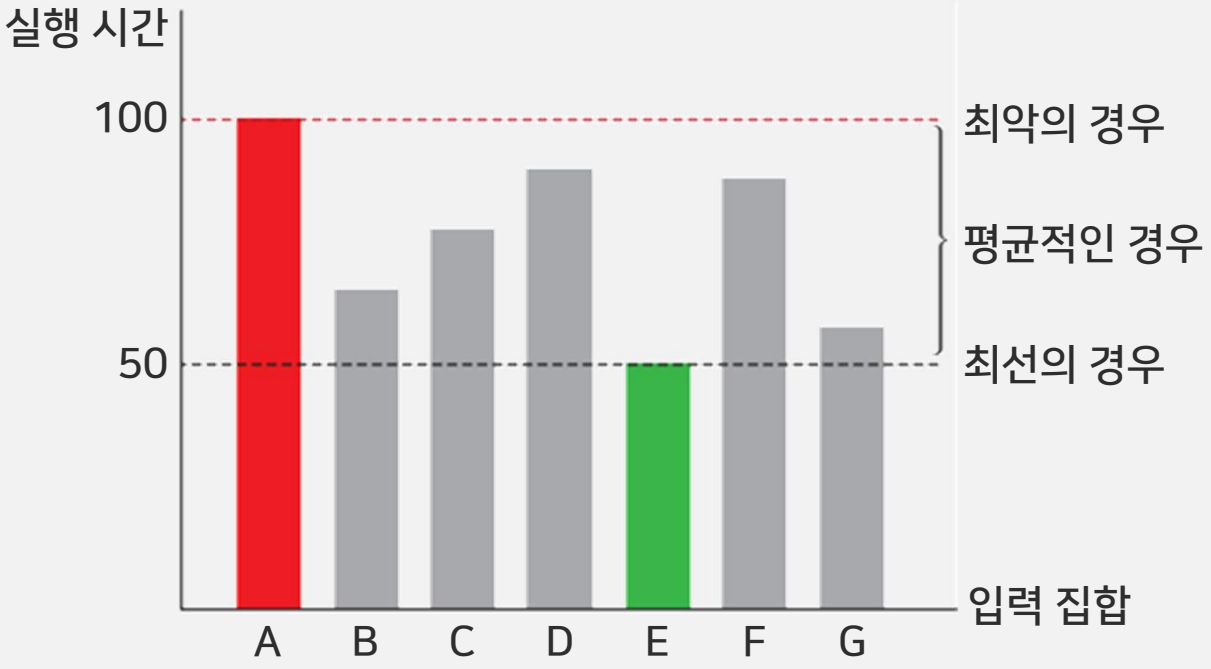
입력에 따라 실행시간이 다를 수 있음

최선의 경우(Best Case)	가장 빠른 경우로 보통 의미 없음
평균의 경우(Average Case)	평균적인 경우로 계산이 어려움
최악의 경우(Worst Case)	가장 늦은 경우로 중요하게 널리 사용

예 비행기 관제업무, 게임, 로보틱스

빅오

최선, 평균, 최악의 경우



같은 알고리즘도 입력의 종류에 따라
처리시간이 크게 달라질 수 있음

순환 알고리즘

순환 알고리즘의 의미

알고리즘이나 함수가 수행도중 자기자신을 다시 호출하는 기법임

정의자체가 순환적으로 되어 있는 경우에 적합함

팩토리얼 구하기

피보나치 수열

이항 계수

하노이의 탑

이진 탐색

$$n! = \begin{cases} 1 & n = 1 \\ n * (n-1)! & n > 1 \end{cases}$$

$$fib(n) \begin{cases} 0 & if\ n = 0 \\ 1 & if\ n = 1 \\ fib(n-2) + fib(n-1) & otherwise \end{cases}$$

순환 알고리즘

팩토리얼

예 n=3 일 때

n=3
```
def factorial(n) :
    if n == 1 : return 1
    else : return n * factorial(n - 1)
```

n=2
```
def factorial(n) :
    if n == 1 : return 1
    else : return n * factorial(n - 1)
```

n=1
```
def factorial(n) :
    if n == 1 : return 1
    else : return n * factorial(n - 1)
```

① ② ③ 1반환 ④ 2반환 ⑤ 6반환

참고

$$n! = \begin{cases} 1 & n = 1 \\ n * (n-1)! & n > 1 \end{cases}$$

factorial(3) = 3 * factorial(2)
= 3 * 2 * factorial(1)
= 3 * 2 * 1
= 3 * 2
= 6

순환 알고리즘

팩토리얼

순환 구조

$n! = n * (n-1)!$

반복 구조

$n! = n * (n-1) * (n-2) * \cdots * 1$

순환 : $O(n)$ (Recursion)

- 순환 문제에서 자연스러운 방법임
- 함수 호출의 오버헤드

반복 : $O(n)$ (Iteration)

- 순환 문제에서 프로그램 작성이 어려울 수 있음
- for나 while문을 이용함
- 수행속도가 빠름

대부분의 순환은 반복으로 바꾸어 작성할 수 있음

순환 알고리즘

거듭제곱

순환(Recursion) | 반복(Iteration)

```
def power(x, n) :
    if n == 0 : return 1
    elif (n % 2) == 0 :                    # n이 짝수
        return power(x*x, n//2)            # 정수의 나눗셈
    else :
        return x * power(x*x, (n-1)//2)    # n이 홀수
```

순환 알고리즘

거듭제곱

순환(Recursion) | 반복(Iteration)

```
def power_iter(x, n) :                # 반복으로 xn을 구하는 함수
    result = 1.0
    for i in range(n):                # 루프: n번 반복
        result = result * x
    return result
```

순환 알고리즘

거듭제곱

시간 복잡도

- 순환 알고리즘의 시간 복잡도 $O(\log_2 n)$

  n이 2의 제곱이라면 문제의 크기가 절반씩 줄어듦

  $$2^k \rightarrow 2^{k-1} \rightarrow \cdots 2^2 \rightarrow 2^1 \rightarrow 2^0$$

- 반복 알고리즘의 시간 복잡도 $O(n)$

```
C:\WINDOWS\system32\cmd.exe
Fast Power(2,500)... 3.273390607896142e+150
Slow Power(2,500)... 3.273390607896142e+150
Fast Power... 0.20188379287719727
Slow Power... 1.8539538383483887
```

두 알고리즘의 결과 동일

순환 함수를 이용한 처리시간(10만 회)

반복 함수를 이용한 처리시간(10만 회)

순환 알고리즘

피보나치 수열

순환(Recursion) | 반복(Iteration)

```
def fib(n) :                          # 순환으로 구현한 피보나치 수열
    if n == 0 : return 0              # 종료조건
    elif n == 1 : return 1            # 종료조건
    else :
        return fib(n-1) + fib(n-2)    # 순환호출
```

참고

$$fib(n) \begin{cases} 0 & n = 0 \\ 1 & n = 1 \\ fib(n-2) + fib(n-1) & otherwise \end{cases}$$

순환 알고리즘

피보나치 수열

순환(Recursion) | 반복(Iteration)

```
def fib_iter(n) :                    # 반복으로 구현한 피보나치 수열

    last = 0
    current = 1
    for i in range(2, n+1) :         # 반복 루프
        tmp = current
        current += last
        last = tmp
    return current
```

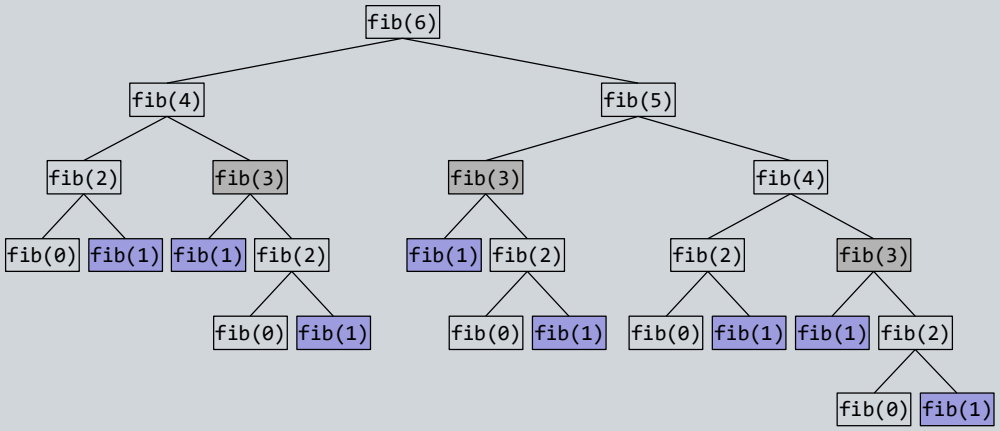
순환 알고리즘

피보나치 수열

시간 복잡도

- 순환 알고리즘의 시간 복잡도 $O(2^n)$

같은 항을 중복 계산

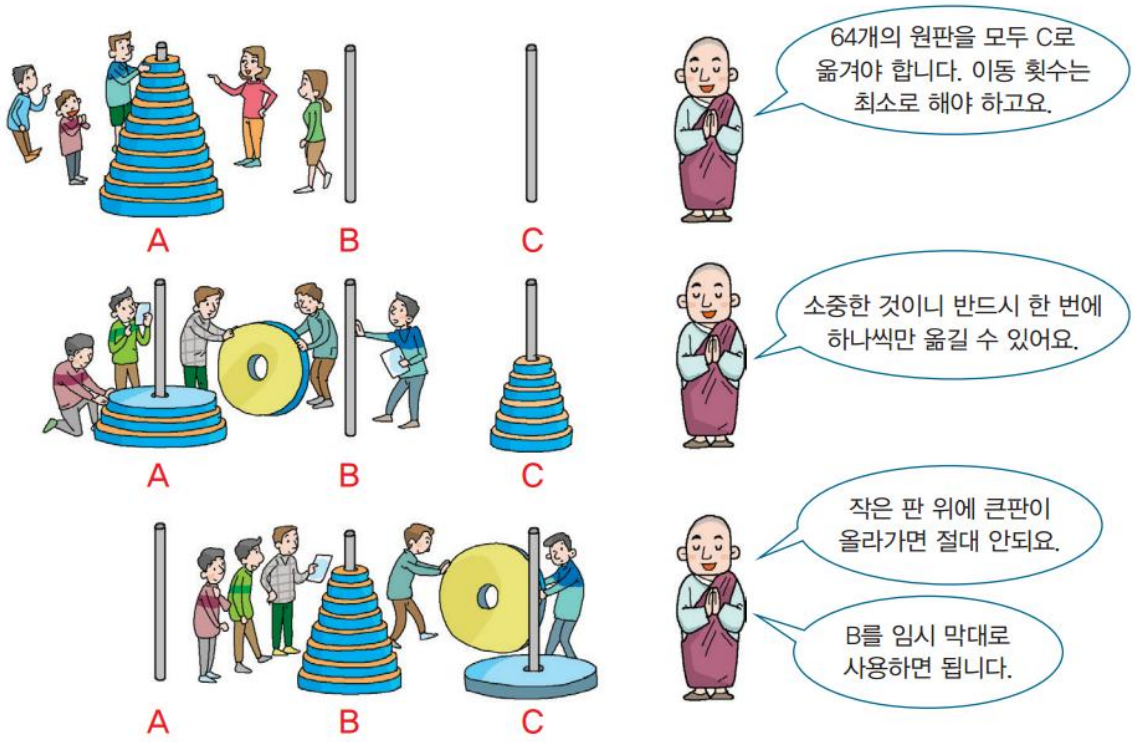


- 반복 알고리즘의 시간 복잡도 $O(n)$

순환 알고리즘

하노이탑

하노이탑 문제란?

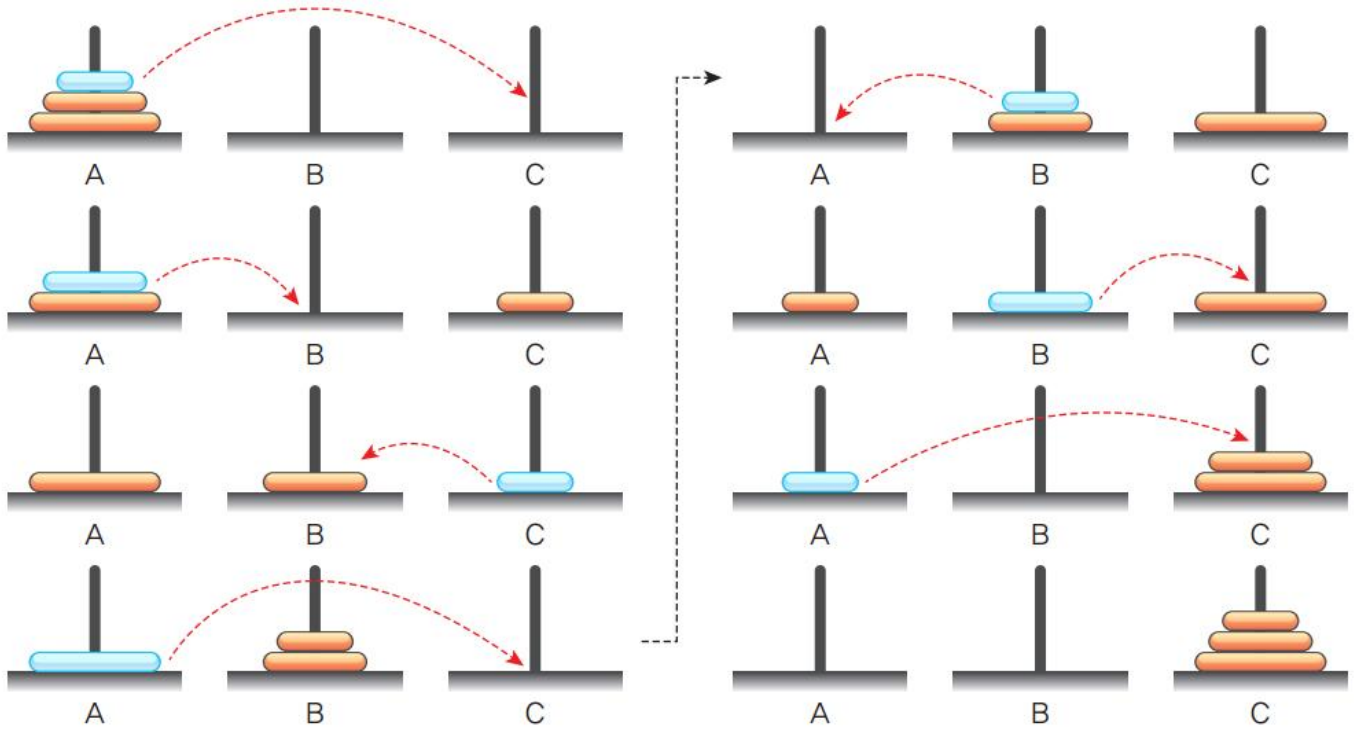


<이미지 출처: 최명규, 천인국. '파이썬으로 쉽게 풀어쓴 자료구조'. 생능출판사>

순환 알고리즘

하노이탑

예 n=3 일 때



<이미지 출처: 최명규, 천인국. '파이썬으로 쉽게 풀어쓴 자료구조'. 생능출판사>

순환 알고리즘

하노이탑

순환 알고리즘 이용

```
def hanoi_tower(n, fr, tmp, to) :                # Hanoi Tower 순환 함수

    if (n == 1) :                                # 종료 조건
        print("원판 1: %s --> %s"% (fr,to))      # 가장 작은 원판을 옮김
    else :
        hanoi_tower(n - 1, fr, to, tmp)          # n-1개를 to를 이용해 tmp로
        print("원판 %d: %s --> %s"% (n,fr,to))   # 하나의 원판을 옮김
        hanoi_tower(n - 1, tmp, fr, to)          # n-1개를 fr를 이용해 to로
```

```
hanoi_tower(4, 'A', 'B', 'C') :                  # 4개의 원판이 있는 경우
```

순환 알고리즘

하노이탑

순환 알고리즘 실행 결과

```
C:\WINDOWS\system32\cmd.exe
원판 1: A --> B
원판 2: A --> C
원판 1: B --> C
원판 3: A --> B
원판 1: C --> A
원판 2: C --> B
원판 1: A --> B
원판 4: A --> C
원판 1: B --> C
원판 2: B --> A
원판 1: C --> A
원판 3: B --> C
원판 1: A --> B
원판 2: A --> C
원판 1: B --> C
```

원판의 이동

예 1번 원판을 A에서 B로 이동함

순환 알고리즘

하노이탑

하노이탑 시간 복잡도

$$
\begin{aligned}
T(n) &= 2T(n-1)+1 \\
&= 2(2T(n-2)+1)+1 \\
&= 2(2(2T(n-3)+1)+1)+1 \\
&= 2^{n-1}T(1)+\cdots \\
&= 2^{n-1}+\cdots \\
&= O(2^n)
\end{aligned}
$$

학습평가

번호	문제	보기	정답	해설
1	문제 해결을 위한 단계적 절차를 무엇이라 하는가?	① 자료구조 ② 자료형 ③ 알고리즘 ④ 복잡도		
2	알고리즘을 기술하기 위한 방법이 아닌 것은?	① 흐름도 ② 측면도 ③ 자연어 ④ 유사코드		
3	추상 자료형에 대한 설명으로 옳지 않은 것은?	① 사용자는 추상 자료형이 어떻게 구현되었는지 정확히 알아야 이용할 수 있다. ② 사용자는 추상 자료형이 제공하는 연산만을 사용할 수 있다. ③ 사용자는 추상 자료형을 어떻게 사용하는지 알아야 한다. ④ 사용자는 추상 자료형 내부의 데이터를 접근할 수 없다.		
4	시간 복잡도 함수는 무엇을 나타내는가?	① 차지하는 메모리 크기 ② 입력 데이터 개수 ③ 연산 횟수 ④ 컴파일 시간		
5	순환 알고리즘 설명 중 옳지 않은 것은?	① 반복을 이용하는 것보다 효율적 ② 문제 자체가 순환적으로 정의된 경우에 적합 ③ 간접적으로 시스템 스택을 사용 ④ 순환이 될 때마다 문제의 크기가 감소		

학습정리

자료구조와 알고리즘의 개념

자료구조와 알고리즘의 정의 및 필요성

- 자료구조는 컴퓨터에서 자료를 정리하고 조직화하는 다양한 구조이다.
- 선형 자료구조는 리스트, 스택, 큐, 덱이 있고 비선형 자료구조는 트리, 그래프가 있다.
- 알고리즘은 문제 해결을 위한 절차를 순서대로 명확하게 나타낸 것이다.
- 알고리즘 기술방법은 자연어, 흐름도, 유사코드, 특정언어를 사용하는 방법이 있다.
- 알고리즘의 조건은 입력, 출력, 명확성, 유한성, 유효성이다.

추상자료형

- 추상 자료형은 데이터 타입을 추상적(수학적)으로 정의하며, 핵심적인 구조나 동작에만 집중한다.

알고리즘의 성능 분석 및 순환 알고리즘

- 알고리즘 성능 분석으로 실행시간 측정은 알고리즘의 실제 실행 시간을 비교 측정하는 것이다.
- 알고리즘의 효율성을 이론적으로 분석하는 방법으로 시간 복잡도와 공간 복잡도 분석이 있다.
- 빅오 표기법은 연산의 횟수를 대략적(점근적)으로 표기한 것이다.
- 순환 알고리즘은 알고리즘 수행 중 자기자신을 다시 호출하여 문제를 해결하는 기법이다.